

Module 1 Assignment

Course: PCCST501 – Computer Networks

Student Name: David Jorin

RollNo: 22

Class: CSEA

How the Internet Delivers a Web Page

Website chosen: Google (www.google.com)

Introduction

This report explains, step by step, everything that happens inside the network when a user types www.google.com into a browser and presses Enter. Every concept used below — protocol layering, DNS, HTTP, TCP, and the client-server model — is explained using the five-layer TCP/IP protocol suite that governs communication on the Internet.

1. TCP/IP Protocol Stack

The TCP/IP protocol suite is a hierarchical set of protocols organized into five layers. Each layer receives services from the layer below it and provides services to the layer above it. When Google's web page is requested, all five layers are involved at the two end hosts (the user's computer and Google's server), while intermediate routers use only the bottom three layers and switches use only the bottom two.

Figure 1: The Five-Layer TCP/IP Protocol Stack

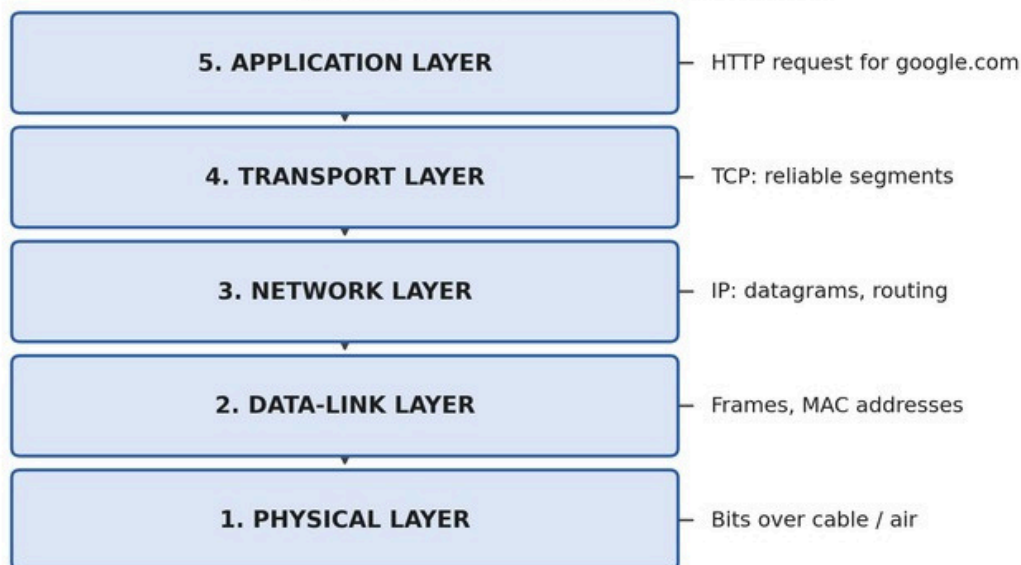


Figure 1: The Five-Layer TCP/IP Protocol Stack

Role of Each Layer

Layer	Unit of Data	Role in Loading google.com
Application	Message	Runs the HTTP protocol; the browser (client) builds a request for Google's home page and the web server (server) builds the response.
Transport	Segment	TCP takes the HTTP message, adds source/destination port numbers (client's ephemeral port and server's port 80), and ensures reliable, ordered, error-free delivery.
Network	Datagram	IP adds the source and destination host addresses and is responsible for routing the datagram from the user's host to Google's server, hop by hop.
Data link	Frame	Adds the physical (MAC) addresses of the current hop and moves the datagram across one single link (e.g., the user's LAN or Wi-Fi).
Physical	Bits	Converts the frame's bits into electrical, optical, or radio signals and transmits them over the transmission medium.

As the message travels down the stack at the sender, each layer wraps the data from the layer above with its own header — this is called encapsulation. At Google's server, each layer removes its corresponding header as the message moves up the stack — this is called decapsulation.

2. Role of DNS in Converting the Domain Name to an IP Address

A browser cannot connect to a server using a name like `www.google.com`; the underlying TCP/IP protocols need a numeric IP address. The Domain Name System (DNS) is a client-server application that maps human-friendly names to IP addresses, similar to how a telephone directory maps a name to a telephone number.

When the user types the URL, the browser first calls a DNS resolver (client). The resolver contacts a local DNS server. If that server does not know the mapping, the query is passed on — first to a root server, then to the top-level-domain server responsible for the `.com` domain, and finally to Google's authoritative DNS server, which returns the actual IP address. Once an address is learned, it is stored in cache memory for a limited time (defined by the Time to Live, or TTL) so that repeated requests for `google.com` do not need to repeat the whole lookup.

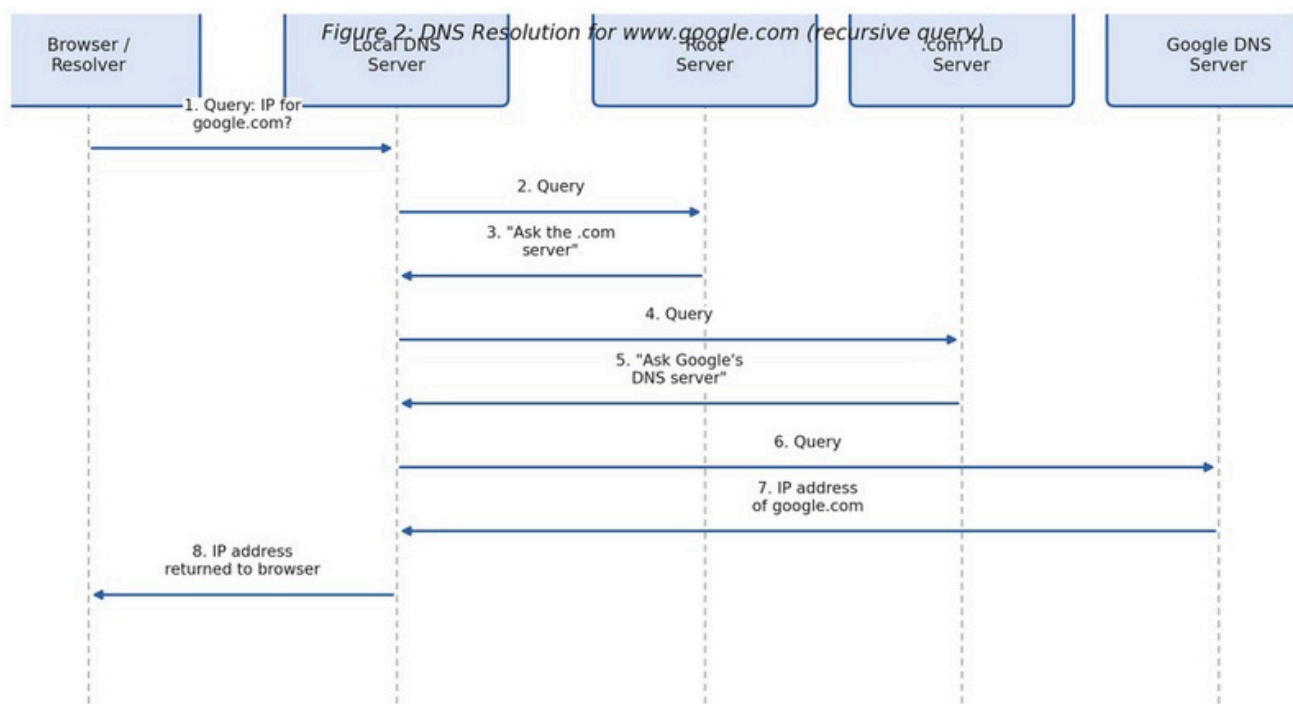


Figure 2: DNS Resolution for *www.google.com* (recursive query)

3. How HTTP Requests and Receives the Web Page

Once the browser has Google's IP address, it uses the HyperText Transfer Protocol (HTTP) to fetch the actual web page. HTTP works on the client-server model: the browser (HTTP client) sends a request message and the web server (HTTP server, listening on the well-known port 80) sends back a response message.

The browser opens a TCP connection to Google's server and sends a request message that has a request line, header lines, a blank line, and (usually empty) body. For loading the Google home page, the request line uses the GET method, for example: GET / HTTP/1.1, followed by header lines describing what content types the browser accepts. Google's server responds with a status line (for example, HTTP/1.1 200 OK), response headers describing the content type and length, a blank line, and finally the body, which contains the HTML of the search page. Because the Google home page also references images, scripts, and style sheets, the browser issues several more HTTP requests — one for each additional object. HTTP version 1.1 uses a persistent connection by default, so all of these objects can be retrieved over a single TCP connection instead of opening and closing a new connection for every object, which saves time and resources.

4. How TCP Provides Reliable Communication for HTTP

HTTP itself has no way of guaranteeing that data arrives correctly, so it depends entirely on the Transport Layer's TCP protocol. Unlike UDP, TCP is connection-oriented: before any HTTP request can be sent, the client and server must first establish a logical connection between themselves by exchanging a small number of handshake messages. This handshaking phase agrees on parameters such as buffer sizes before real data is exchanged.

- Connection establishment: At least three handshake messages are exchanged between the browser and Google's server before the HTTP request is actually sent.
- Reliable, ordered delivery: TCP treats the HTTP message as a stream of bytes and numbers them, so the receiver can detect missing or out-of-order bytes and request that they be resent — this is TCP's error control.
- Flow control: TCP matches the sending rate of Google's server to the receiving rate of the user's device, so a slower device is not overwhelmed.

- Congestion control: TCP reduces its sending rate when the network itself is congested, which prevents data loss on busy links.
- Connection termination: After the requested objects have been transferred, at least three more handshake messages are used to close the TCP connection gracefully.

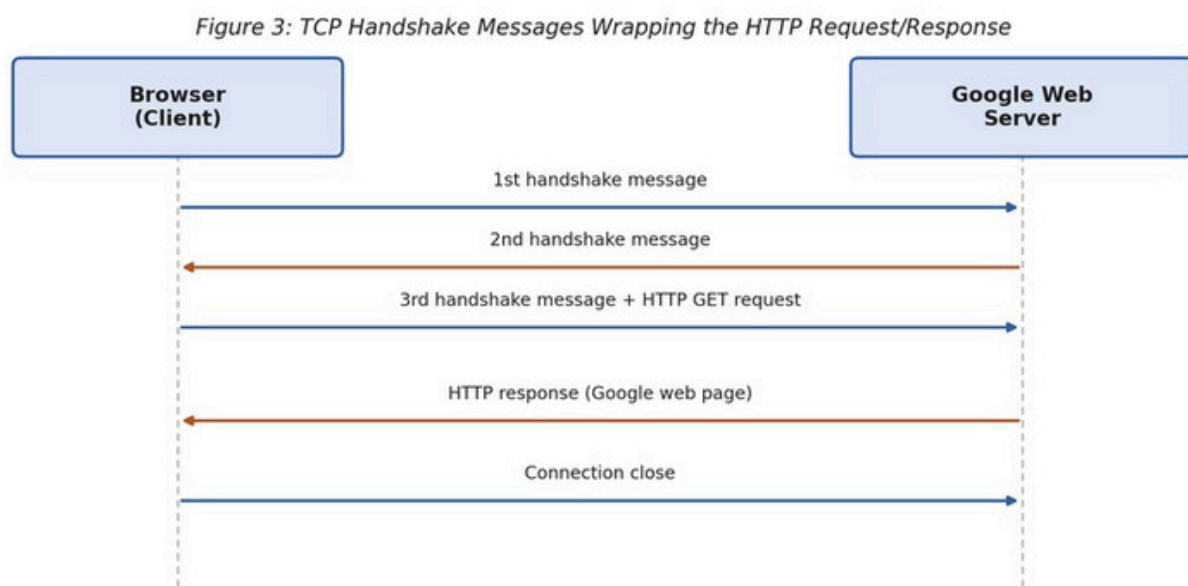


Figure 3: TCP Handshake Messages Wrapping the HTTP Request/Response

5. Client–Server or Peer-to-Peer Architecture?

Loading a Google web page follows the client-server paradigm, not the peer-to-peer paradigm. In the client-server model, the service provider is an application program called the server process, which runs continuously and waits for a client process to connect and request a service; the client is started only when the user needs the service and stops once the response is received.

This exactly matches how Google works: Google's web servers run all the time, ready to accept requests from anywhere in the world, while the user's browser (the client) is started only when the user wants to search or view a page, sends a request, receives the response, and then becomes idle. The communication load is concentrated on Google's side, so Google must operate large, powerful server farms — a typical characteristic of the client-server paradigm. This is different from a peer-to-peer application (such as file-sharing software), where every computer can act as both a client and a server and there is no single always-on service provider.

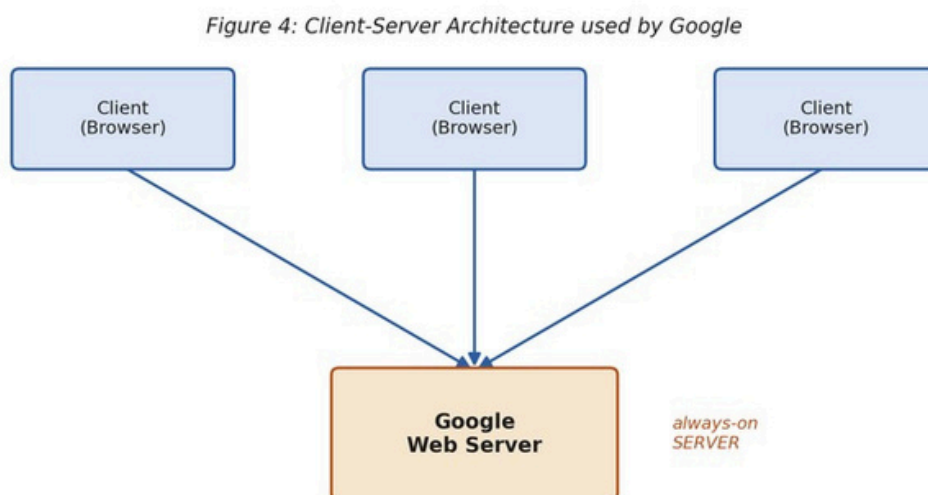


Figure 4: Client-Server Architecture used by Google

6. HTTP vs FTP – Key Differences

Although both HTTP and FTP are standard client-server application-layer protocols that run over TCP, they were designed for different purposes and behave very differently.

Feature	HTTP	FTP
Purpose	Requests and delivers web pages and their embedded objects (text, images, scripts).	Transfers files of any type between a client and a server.
Connections used	Uses a single TCP connection (persistent by default in HTTP/1.1) for both control-type information and data.	Uses two separate TCP connections: a control connection (port 21) that stays open for the whole session, and a data connection (port 20) that opens and closes for each file transfer.
Message style	Simple request/response pair: a request line, headers, blank line, and body.	Uses text commands (e.g., RETR, STOR, LIST) on the control connection, each followed by a three-digit numeric response.
Statefulness	Originally stateless; state across requests is added artificially using cookies.	Inherently stateful — the control connection remains open and keeps track of the login session and current directory.
Typical use with Google	Used to fetch the Google search page, its images, and scripts.	Not used for delivering the Google home page; FTP is meant for bulk file transfer, not for interactive web browsing.

7. How Cookies Improve the User Experience

HTTP was originally designed to be stateless — after a server sends its response, it does not remember anything about the client. This is a problem for modern websites like Google, which need to remember user preferences across multiple visits. Cookies solve this problem: when a server sends a response, it can include a Set-Cookie header containing a small piece of information. The browser stores this cookie and automatically sends it back inside the Cookie header on every future request to the same server, allowing the server to recognize the returning user without asking them to log in or set preferences again.

Real-world example: When a user visits Google and changes a setting such as the preferred interface language or the SafeSearch filter level, Google's server sends a cookie back with the response that stores this choice. The next time the same browser sends a request to Google — even for a completely different search — that cookie is automatically included in the request. Google's server reads the cookie and immediately applies the saved language and filter settings without the user having to reconfigure them, giving a faster and more personalized browsing experience.

8. Complete Communication Flow: Browser to Web Server and Back

Bringing all the previous steps together, the diagram below shows the full sequence of events, from the moment the user presses Enter to the moment the Google page appears on screen.

Figure 5: End-to-End Flow from Browser to Web Server and Back

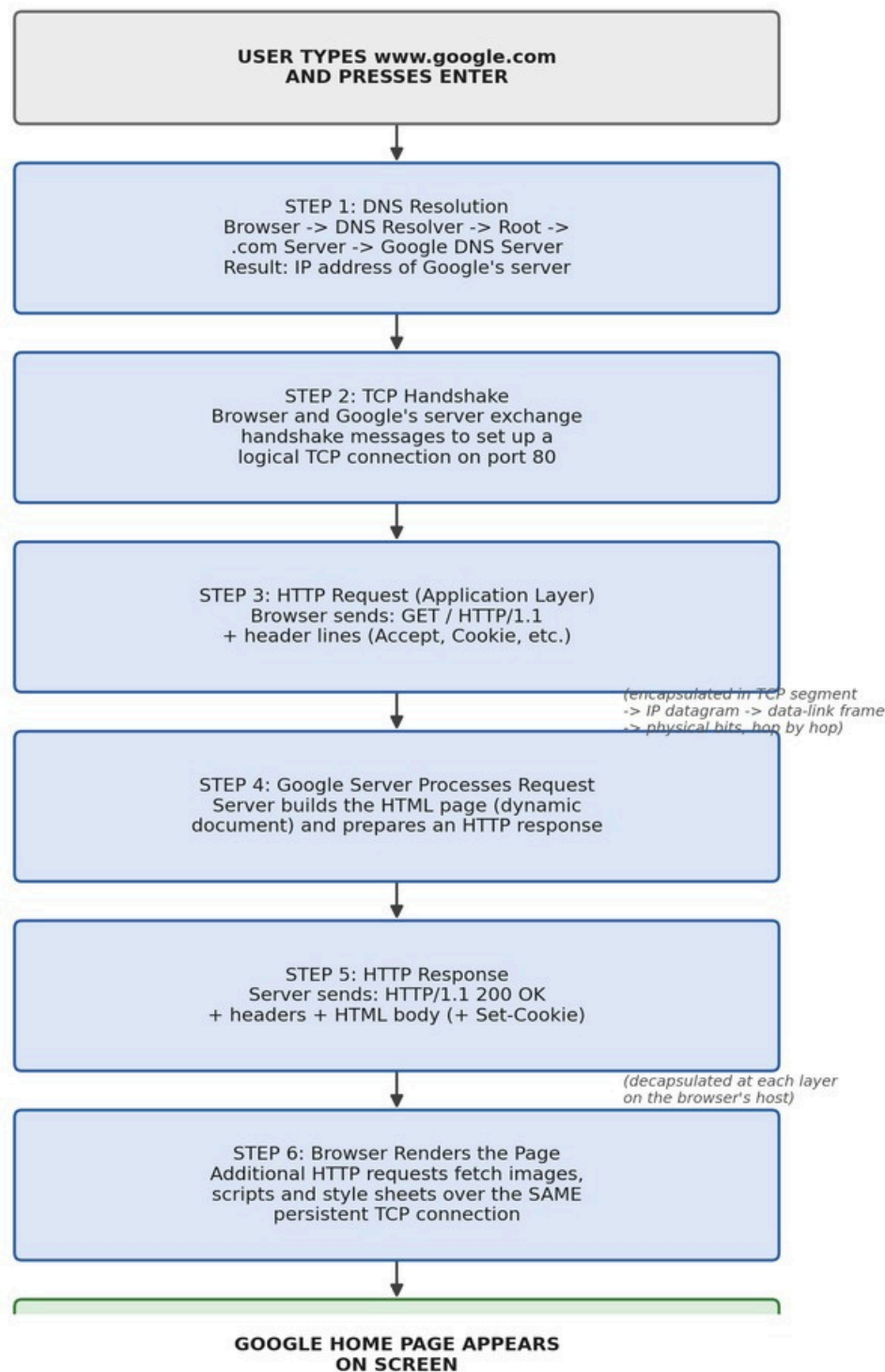


Figure 5: End-to-End Flow from Browser to Web Server and Back

9. Conclusion

This report has traced the complete journey of a single request to google.com, from the application layer down to the physical layer and back again. Application-layer protocols are the reason the Internet feels simple to use even though the underlying system is extremely complex: DNS hides numeric addressing behind memorable names, HTTP defines a common language for browsers and web servers to exchange pages, and TCP silently guarantees that data arrives complete and in order. The client-server paradigm allows a single, always-available service like Google to serve millions of users at once, while cookies add the personal touch of memory to an otherwise stateless protocol. Without these application-layer protocols working together through the layered TCP/IP model, everyday tasks such as searching the web would not be possible. Understanding this layered process is therefore essential to understanding how the modern Internet truly works.